

Software Requirements Specification (SRS)

LogiVis Lark Messenger Integration — Phase 1: Lark Communication POC

Version 1.0 | Status: Development / POC | Backend: .NET | Frontend: Existing Angular LogiVis | Communication Platform: Lark

1. 1. Purpose

Phase 1 is a technical proof of concept to verify that LogiVis can communicate with Lark through the Lark Bot/Open API. The POC establishes two-way technical communication between LogiVis, the .NET backend, and Anthony's Lark account.

2. 2. Phase 1 Objectives

Verify the existing LogiVis Lark authentication.

Determine the identity information available for the authenticated user.

Configure the Lark Bot and required API permissions.

Verify that the .NET backend can send a message to Anthony's Lark account.

Verify that the .NET backend can receive Lark webhook events.

Determine whether incoming events contain sufficient sender, message, and conversation identifiers.

Capture the actual Lark webhook payload for subsequent implementation phases.

3. 3. Existing System and Constraints

LogiVis is an Angular application.

LogiVis already uses Lark authentication for users.

The existing authentication must be inspected rather than assumed to provide messaging authorization.

The integration backend shall use .NET.

Phase 1 does not introduce another user login.

Phase 1 does not use WhatsApp.

4. 4. Scope

In Scope

Inspection of existing Lark authentication.

Lark Bot configuration.

Lark API authentication and minimum required permissions.

.NET Lark integration service.

Outbound self-message test.

Lark webhook endpoint.

Webhook verification/challenge.

Inbound reply test.

Logging of actual Lark event payloads.

Basic error handling.

Out of Scope

Messenger UI.

Reusable Web Component.

Google Sheets.

Google Drive.

AI.

Jira.

Database.

Media processing.

Advanced conversation management.

Notifications.

5. 5. Proposed Architecture

Outbound: Existing LogiVis → .NET Backend → Lark Bot/Open API → Anthony's Lark Account.

Inbound: Anthony's Lark Account → Lark Event/Webhook → .NET Backend.

No Lark credentials or secrets shall be placed in the Angular frontend.

6. 6. Authentication and Authorization

LogiVis already authenticates users through Lark. The implementation shall first inspect the existing authentication flow and determine what Lark identity information is available to the Angular application and .NET backend.

The POC shall verify whether the existing authentication is sufficient to identify the logged-in Lark user and whether the backend has the authorization required to send a message through the Lark Bot/Open API.

If additional Lark permissions, scopes, tokens, or authorization are required, the minimum required mechanism shall be documented and implemented only as necessary.

7. 7. Functional Requirements

FR-01 — Authentication Inspection

Inspect the existing LogiVis Lark authentication and document available identity information.

FR-02 — Bot Configuration

Configure a Lark Bot application with the minimum required permissions.

FR-03 — Outbound Messaging

The .NET backend shall send a test message through the Lark Bot/Open API to Anthony's Lark account.

FR-04 — Recipient

Anthony's Lark account shall be the fixed recipient for the Phase 1 POC.

FR-05 — Webhook

The .NET backend shall expose a Lark webhook endpoint for supported events.

FR-06 — Webhook Verification

The webhook shall support Lark's required verification/challenge process.

FR-07 — Inbound Events

The backend shall receive and log a reply/event from Lark.

FR-08 — Identity

The POC shall determine what sender identity information is provided by Lark.

FR-09 — Message Identification

The POC shall determine available event, message, and chat/conversation identifiers.

FR-10 — Raw Payload

The complete real Lark webhook payload shall be logged for analysis.

FR-11 — Error Handling

API and webhook errors shall be logged clearly.

8. 8. Test Case 1 — Self Message

Use one Lark account for both sides of the initial test. No colleague is required.

Flow: LogiVis → .NET → Lark Bot/API → Anthony's Lark Account.

Expected result: the test message appears in Anthony's Lark account.

9. 9. Test Case 2 — Lark Reply

Anthony replies using normal Lark Messenger.

Flow: Anthony → Lark → Lark Event/Webhook → .NET.

Expected result: the .NET webhook receives the event and logs the actual payload.

10.10. Webhook Data Investigation

Where available, inspect and record:

Event ID

Message ID

Chat ID

Sender ID

Sender name

Timestamp

Message type

Message content

Reply-to message ID

Raw event payload

Field names and payload structure shall be based on the actual Lark event received during the POC, not assumptions.

11.11. Logging

Phase 1 does not require Google Sheets or a database. Application logging is sufficient.

The log should clearly show the event/message identifiers, sender information, timestamp, message type/content where available, and the complete raw webhook payload.

12.12. Security

Lark App Secret and tokens shall remain server-side.

Google credentials are not required in Phase 1.

No secrets shall be embedded in Angular or the future Web Component.

Webhook requests shall be validated according to Lark requirements.

13.13. Error Handling

Missing Lark configuration.

Invalid credentials or token.

Insufficient API permissions.

Invalid recipient.

Lark API failure.

Webhook verification failure.

Invalid or unsupported webhook payload.

14.14. Implementation Sequence

1. Inspect the existing LogiVis Lark authentication.
2. Identify available Lark user identity information.
3. Configure the Lark Bot and minimum required permissions.
4. Implement the minimum .NET Lark API service.
5. Send a self-test message to Anthony's Lark account.
6. Configure the Lark webhook/event subscription.
7. Verify the webhook challenge.
8. Reply from Lark and capture the webhook event.
9. Inspect and document the real payload.
10. Confirm whether existing authentication is sufficient for the future Messenger.

15.15. Acceptance Criteria

The existing LogiVis Lark authentication has been inspected and its available identity information is documented.

The Lark Bot is configured with the required minimum permissions.

A message can be sent from the .NET backend to Anthony's Lark account.

Anthony can reply using normal Lark Messenger.

The reply reaches the .NET webhook.

The actual Lark event payload is captured in the application log.

Available sender, message, event, and chat identifiers are identified.

It is determined whether the existing LogiVis authentication is sufficient for the future Messenger or what additional authorization is required.

No WhatsApp dependency exists.

16.16. Phase 2 Preview

Only after Phase 1 succeeds: build the reusable Messenger Web Component, integrate it into Angular LogiVis, add Google Sheets/Drive storage, and implement the broader messaging workflow.

17.17. Guiding Principle

Prove the Lark plumbing before building the Messenger. Use the real Lark API responses and webhook payloads as the source of truth for subsequent design.